

OLD BUZZ — Authentication & Community

Membership Flow Audit

Type	Read-only source audit (no application code changed)
Date	2026-09-21
Subject	OLD BUZZ (buzz/buzz , reference implementation) compared with SWF BUZZ (buzz/swf buzz)
Method	Source tracing (call sites, SQL, migrations). Documentation was used only as a pointer, never as evidence.

Confidence labels: **CONFIRMED** = read directly in source. **INFERRED** = follows from confirmed code but the last hop was not read. **UNCLEAR** = not confirmed from the available source.

Path convention: OLD: = buzz/buzz/..., SWF: = buzz/swf buzz/... . Line numbers are as of the audited checkout.

1. Executive Summary

The hypothesis is incorrect in its causality. Invite creation does *not* generate a key pair, and accepting an invite does *not* create a new identity. The actual order in OLD BUZZ is:

- The desktop app generates a Nostr secp256k1 key pair on first launch**, before any community or invite exists (OLD: desktop/src-tauri/src/app_state.rs:187-199, 627-650). It is stored in the OS keyring (default build feature) with a 00600 identity.key file as fallback.
- The owner mints an invite.** The invite is a random 32-byte secret, v2.<base64url> . Only its SHA-256 hash is stored (OLD: crates/buzz-db/src/store/relay_invite.rs:99-135). It contains no public key, no private key, and no user reference. It is bound only to a community (tenant) and to created_by .
- The invited user's existing key signs a NIP-98 HTTP request** to POST /api/invites/claim . The relay then inserts (community_id, pubkey, role='member', added_by='invite') into relay_members (OLD: relay_invite.rs:331-332).

So the *outcome* in the hypothesis is true (a public key ends up bound to a community as a member), but the key **pre-exists** the invite. Invites only *authorize a pubkey to join*; they never create one.

What identifies a human: the Nostr public key (hex), always. There is no email, password, OAuth, Okta or account system in the relay path. Every authentication step is a signature by the user's private key (NIP-42 for WebSocket, NIP-98 for HTTP).

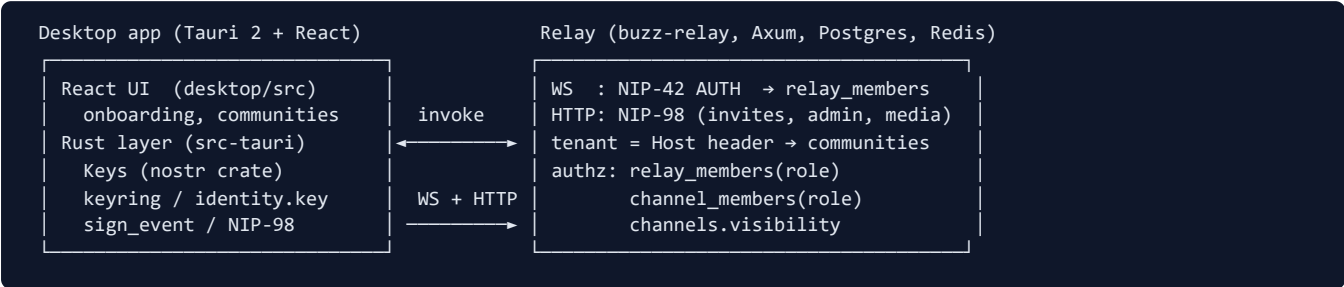
Two more findings that change planning:

- Community creation is not a user action on the relay.** Communities are tenants keyed by HTTP Host . They are provisioned by a deployment *operator* (POST /operator/communities , gated by RELAY_OPERATOR_PUBKEYS) or seeded. The desktop "create a hosted community" path goes through an external **BuilderLab** SaaS account, whose backend is **not in this repository** (**UNCLEAR** beyond the desktop client).
- A community owner does not automatically have channel access.** Community role (relay_members) and channel role (channel_members) are separate tables, and the message read/write gate consults only channel membership or channel visibility.

NIP-FI (OIDC/JWKS federated identity) exists in buzz-auth but is dormant. No relay, desktop or other-crate source references it. OLD BUZZ therefore has no working "login with an identity provider" path to replicate; SWF's Okta path is new work, not a port.

SWF BUZZ state: a new `swf-buzz-backend` already implements Okta → session → Application User → community membership → invites (HTTP, no Nostr). The frontend is mid-migration: the old pubkey/Nostr paths and the new HTTP paths coexist (\$23).

2. OLD BUZZ Architecture Relevant to Authentication



Layer	Responsibility	Evidence
Tauri Rust	Generates/holds the key, signs events, builds NIP-98 headers	<code>app_state.rs</code> , <code>commands/identity.rs</code>
Relay tenancy	A "community" is a row in <code>communities</code> resolved from the request <code>Host</code>	<code>api/invites.rs:240</code> (<code>bind_community</code>), migration 0001
Relay authn	NIP-42 (WS), NIP-98 (HTTP)	<code>handlers/auth.rs</code> , <code>api/invites.rs:230-265</code>
Relay authz	<code>relay_members</code> (community), <code>channel_members</code> + <code>channels.visibility</code> (channel)	migrations 0001

Confidence: **CONFIRMED**.

3. Human User Identity Model

- **Primary identity = Nostr public key** (hex). It is the primary key of every membership row: `relay_members` (`community_id`, `pubkey`, `role`, `added_by`), `PK` (`community_id`, `pubkey`) (OLD: `migrations/0001_initial_schema.sql:574-582`) and `channel_members` (`community_id`, `channel_id`, `pubkey`, `role`, ...) (0001:132-145). **CONFIRMED**.
- **The private key is required** to authenticate (NIP-42 AUTH event) and to sign every write event and every NIP-98 HTTP call (OLD: `desktop/src/shared/api/invites.ts:47-62`, `handlers/auth.rs:87-92`). **CONFIRMED**.
- The same pubkey can be a member of many communities; each membership is a separate row scoped by `community_id`. **CONFIRMED**.
- **Optional overlay:** a BuilderLab account can be bound to a pubkey (`HostedNostrIdentity`, error codes `identity_already_bound` / `pubkey_already_bound`) for *hosted community creation only* (OLD: `desktop/src/features/communities/hostedCommunityApi.ts`). It does not replace the pubkey identity. The BuilderLab side is **UNCLEAR** (external).
- **Agents** are ordinary Nostr participants, linked to a human owner via NIP-OA delegation tags (`handlers/auth.rs:134-157`, `219-279`).

4. New User Login Flow (first launch)

There is **no login screen with credentials**. "Login" is "the app already holds a key".

#	Step	Where	Confidence
1	App boots; <code>build_app_state()</code> sets a placeholder ephemeral key (or <code>BUZZ_PRIVATE_KEY</code> env override, which wins)	<code>app_state.rs:187-199</code>	CONFIRMED
2	<code>resolve_persisted_identity()</code> runs in <code>setup(): env → keyring → identity.key file → generate + save</code>	<code>app_state.rs:251-297, 358</code>	CONFIRMED
3	<code>Keys::generate()</code> runs when neither store has a key (first-ever launch)	<code>app_state.rs:627-650</code> (<code>load_file_or_generate</code>)	CONFIRMED
4	UI stage machine: <code>blocking → keyring-locked → onboarding → ready</code> (machine onboarding), keyed per pubkey in <code>localStorage</code>	<code>desktop/src/features/onboarding/machineOnboarding.ts:1-25</code>	CONFIRMED
5	Onboarding offers: view/backup the key (<code>get_nsec</code> , NIP-49 <code>ncryptsec</code> backup), or import an existing key (<code>import_identity</code>)	<code>commands/identity.rs:191, 243, 337; key_backup.rs</code>	CONFIRMED
6	Profile step publishes a <code>kind:0</code> profile via <code>update_profile</code>	<code>commands/profile.rs:40-80</code>	CONFIRMED
7	Community stage machine: <code>claiming → connecting → profile → team-intro → finalizing → entering</code>	<code>communityOnboarding.tsx:17-28</code>	CONFIRMED

5. Existing User Login Flow

On subsequent launches step 2 finds the key in the keyring (or file) and loads it. Recovery states exist: `keyring-locked` (keyring unreachable, marker present, an ephemeral key is used and the UI asks to unlock and relaunch), and `identity_lost` (marker present but keyring empty) (`app_state.rs:61-91, 507-560`). The user reconnects to the relay by answering the NIP-42 challenge with the same key. **CONFIRMED.**

Sign-out is destructive: it writes a reset sentinel and the next boot wipes the identity (`commands/identity.rs:537-560`). Without a backup, signing out means losing the identity. **CONFIRMED.**

6. Key Generation / Key Storage Analysis

Direct answers to the audit questions:

Question	Answer	Evidence	Conf.
A. Who generates the key?	The desktop Rust layer, locally	<code>app_state.rs:198, 643</code>	CONFIRMED
B. When?	First launch, at <code>setup()</code> . Not at invite creation or claim	<code>app_state.rs:266-297</code>	CONFIRMED
C. Type	<code>secp256k1</code> , <code>nostr::Keys</code>	<code>identity_storage.rs:1</code>	CONFIRMED
D. Nostr key?	Yes (<code>nsec</code> / <code>npub</code> bech32 forms exist)	<code>commands/identity.rs:191</code>	CONFIRMED
E. Storage	OS keyring (<code>system-keyring</code> is a default feature) → fallback <code>identity.key</code> file (<code>00600</code>) → env override → ephemeral. Enum <code>IdentityStorage { Ephemeral, SystemKeyring, LocalFile, Environment }</code>	<code>identity_storage.rs:8-13;</code> <code>Cargo.toml:25</code>	CONFIRMED
F. Does the user see it?	Yes: <code>get_nsec</code> , masked display, encrypted NIP-49 backup and file export	<code>commands/identity.rs:191-337;</code> <code>NsecMaskedDisplay.tsx</code>	CONFIRMED

Question	Answer	Evidence	Conf.
G. Public key sent to relay?	Yes: in every signed event, the AUTH event and NIP-98 headers	handlers/auth.rs:87	CONFIRMED
H. Public key stored in the membership record?	Yes: relay_members.pubkey	migration 0001:574	CONFIRMED
I. Does the invite contain a public key?	No. 32 random bytes; DB stores only sha256(code) , created_by (creator pubkey, audit only), expiry, max_uses , use_count	relay_invite.rs:109-135 ; migration 0025:18	CONFIRMED
J. Accepting associates an existing identity?	Yes	api/invites.rs:378, 401-406	CONFIRMED
K. Accepting creates a new identity?	No	same	CONFIRMED
L. What creates the membership?	claim_relay_invite → INSERT INTO relay_members ... 'member', 'invite' inside one transaction with FOR UPDATE on the invite row	relay_invite.rs:222-345	CONFIRMED
M. Role assigned	Always member . The relay_invites.role column is CHECK-constrained to 'member' : "invite links never grant admin"	migration 0025	CONFIRMED
N. How is access authorized after joining?	WS handshake gate enforce_relay_membership , then channel-level checks (§15-16)	handlers/auth.rs:219-241	CONFIRMED

The one place a *new* key is legitimately created around joining: the user may **import a different key** during machine onboarding, *before* the claim. The claim hook's own comment says it "claims the invite with the user's final identity" (OLD: onboarding/useClaimInvite.ts). The relay treats whichever key signs the claim as the joiner.

7. Community Creation Flow

A community is a **tenant**, not a user-created object on the relay.

Path	Mechanism	Evidence	Conf.
Self-hosted / local	Seed script inserts host rows into communities ; RELAY_OWNER_PUBKEY is bootstrapped into relay_members as owner on first startup	config.rs:244-247 ; scripts/seed-local-community.sh (seeded 4 host aliases in this environment)	CONFIRMED
Operator provisioning	POST /operator/communities (also /archive , /unarchive , /availability , /transfer), NIP-98 signed, gated by RELAY_OPERATOR_PUBKEYS ; body {host, initial_owner_pubkey?} . Empty allowlist disables it (fail closed)	handlers/community_provisioning.rs:1-25 ; router.rs:88-104 ; config.rs:249-267	CONFIRMED
Hosted (desktop UI)	User signs in to a BuilderLab account (app.builderlab.xyz/api/goose), binds their pubkey, and BuilderLab provisions <name>.communities.buzz.xyz (limit 5/account)	builderlab.rs:15,22 ; hostedCommunityApi.ts	CONFIRMED (client) / UNCLEAR (server)

8. Community Owner Flow

- **Owner identification:** the `relay_members` row with `role='owner'` for that `community_id` and the owner's pubkey (CHECK: `owner|admin|member`). **CONFIRMED.**
- **Owner login:** identical to every other user (their key, NIP-42). There is no separate owner login. **CONFIRMED.**
- **Owner powers (community level):** mint invites (owner/admin), add members `kind:9030`, remove `9031`, change role `9032`, set workspace icon `9033`. Owners may grant admin; admins may only add members; promoting to owner must use `9032` (`handlers/relay_admin.rs:255-330`). **CONFIRMED.**
- **Operators are above owners:** deployment-level operators can create communities and rotate the owner, but hold no implicit tenant membership row (`config.rs:262-267`). **CONFIRMED.**

9. Invite Creation Flow

```
Owner UI (CommunityInviteDialog / InviteLinkSection)
→ mintInvite()                                desktop/src/shared/api/invites.ts:194-209
→ signRelayEvent(kind 27235, tags u/method/payload/nonce)  invites.ts:47-62
→ POST {relay}/api/invites  Authorization: Nostr <base64 NIP-98 event>
Relay mint_invite()                            OLD: crates/buzz-relay/src/api/invites.rs:284-353
1. authenticate(): bind tenant from Host, verify NIP-98 + replay guard (lines 230-265)
2. role lookup in relay_members; must be owner|admin else 403 (lines 291-304)
3. validate ttl 60s..30d (default 72h), max_uses 1..10000 (lines 63-87)
4. mint_relay_invite(): 32 random bytes → "v2."+b64url; INSERT sha256(code) (relay_invite.rs:99-135)
5. respond {code, expires_at, max_uses, uses_remaining, url: "{http(s)}://{host}/invite/{code}"}
```

All steps **CONFIRMED**. The invite is an **HTTP-API-minted opaque bearer code**, not a Nostr event (`kind:9009` `CREATE_INVITE` is a separate NIP-29 *channel* invite kind, out of scope here).

10. Invite URL / Code Structure

Item	Value	Evidence
Code (v2)	v2. + base64url(32 random bytes), no padding, canonical-checked	<code>buzz-core/src/invite.rs:18-55</code>
Stored	<code>token_hash = sha256(code)</code> (BYTEA 32). Raw code is never stored	<code>invite.rs:58</code> ; migration 0025
URL	<code>https://<community-host>/invite/<code></code> (relay serves a landing page)	<code>api/invites.rs:346-352</code> ; <code>router.rs:240</code>
Deep link	landing page opens <code>buzz://join?relay=<wss-url>&code=<code>[&policy_receipt=...]</code>	<code>web/src/features/invite/ui/InvitePage.tsx:13,248</code> ; <code>desktop/src-tauri/src/deep_link.rs:344-362,630-642</code>
Community binding	Implicit: the unique key is (<code>community_id</code> , <code>token_hash</code>), so a code from community A is Invalid on host B	migration 0025; <code>relay_invite.rs:222-245</code>
Community ID in the code?	No. The community is the request Host	same
Owner pubkey associated?	Only as <code>created_by</code> audit column	<code>relay_invite.rs:129-134</code>

Item	Value	Evidence
Legacy v1	HMAC-signed stateless token {c: community uuid, r: role, e: expiry, n: nonce}.mac, still accepted (drain window); no use limits	api/invites.rs:459-514; invite_token.rs

Confidence: **CONFIRMED**. The landing-page rendering itself (web/) was read only at the two `buzz://join` call sites, so any other landing-page behavior is **UNCLEAR**.

11. Invited User Flow

The suspected flow is only partly right; verified sequence:

```
User B receives https://<host>/invite/<code>
└─ browser landing page (web InvitePage) ┤ buzz://join?relay=wss://<host>&code=<code>
    └─ Rust deep_link.rs parse_join_deep_link ┤ queue_community_deep_link("join")
        └─ TS shared/deep-link.ts drainPendingCommunityDeepLinks
            └─ startCommunityOnboarding(source "deep-link-join", inviteCode) stage="claiming"
                (deep links are persisted BEFORE machine onboarding completes)
Machine onboarding first: key already exists (or user imports one) ← identity comes FIRST
Then useClaimInvite → claimInvite(relayUrl, code, policyReceipt) onboarding/useClaimInvite.ts
    → NIP-98-signed POST /api/invites/claim (signed by B's existing key)
    → relay: joined | already_member | invite_invalid|expired|exhausted
Then stage "connecting" → WS connect + NIP-42 AUTH → relay_members gate passes
Then profile → team-intro → finalizing → entering (lands on Welcome channel)
```

CONFIRMED for every hop above. Exception: how the user lands in the Welcome channel (whether membership is created automatically) is **UNCLEAR**; the join-side code for that was not traced.

12. Invite Acceptance / Claim Flow

OLD: `api/invites.rs:361-515` and `relay_invite.rs:222-345`. **CONFIRMED**.

- `authenticate()`: NIP-98 signature + payload hash + replay check; pubkey = the claimer.
- Rate limit: 10 claims per pubkey per 60 s, per community; capped cache (`invites.rs:38-46, 368-373`).
- Optional operator **join policy** (terms/age): a relay-issued HMAC receipt bound to the invite code is required (`invites.rs:391-398, accept_policy` at 199-226).
- v2. prefix → DB-backed path (no fallback to v1). Transaction: `SELECT ... FOR UPDATE ON (community_id, token_hash)`; reject Invalid / Expired; if the pubkey is already a member → `AlreadyMember` (idempotent, does not consume a use); if `use_count >= max_uses` → Exhausted; else `INSERT relay_members ... 'member', 'invite'` ON CONFLICT DO NOTHING, then `use_count++` in the same commit.
- On Joined only: publish NIP-43 `member-added` (8000) and membership-list (13534) events (`invites.rs:424-431`).
- Response `{status, community_id, host, role:"member"}`.

13. Community Membership Creation

Two writers to `relay_members` (both **CONFIRMED**):

Writer	Actor	Role granted
Invite claim	The joiner (self), authorized by the invite secret	<code>member</code> , <code>added_by='invite'</code>
<code>kind:9030 add-member</code>	Owner/admin, by pubkey	<code>member</code> or <code>admin</code> (admin only by owner)

There is **no owner-approval step** after an invite claim: the claimer becomes an active member immediately (`relay_invite.rs:331`). Approval-style gating does not exist in this path. The optional join-policy acceptance is a user-side attestation, not an approval.

14. Role Assignment

Scope	Roles	Assigned by	Evidence
Community (<code>relay_members</code>)	<code>owner</code> , <code>admin</code> , <code>member</code>	<code>invite</code> → <code>member</code> ; <code>9030 add</code> ; <code>9032 change</code> (only path to owner)	<code>migration 0001:574</code> ; <code>relay_admin.rs:288-330</code>
Channel (<code>channel_members</code>)	<code>member_role</code> enum (<code>owner/admin/member/...</code>)	<code>9000 put-user, self-join 9021</code> (open channels → <code>Member</code>), creator becomes owner	<code>side_effects.rs:1950-1985</code> ; <code>channel_authz.rs:1-80</code>
Platform (operator)	<code>allowlist</code> <code>RELAY_OPERATOR_PUBKEYS</code> , <code>fallback RELAY_OWNER_PUBKEY</code>	deployment config	<code>config.rs:249-267</code> ; <code>api/admin/auth.rs:243-275</code>

Last-owner protection exists at channel level (`is_sole_owner`, `channel_authz.rs:63-70`). **CONFIRMED**.

15. Channel Access

- Channels have `visibility` ∈ {`open`, `private`} (migration 0001:29, 77).
- Self-join** (`kind:9021`) **works only for** `open` channels; `private` returns "request an invitation" (`side_effects.rs:1956-1965`). **CONFIRMED**.
- Reads**: REQ handler computes `accessible_channel_ids` for the pubkey (channels the pubkey is a member of, plus open ones) and filters results to it (`handlers/req.rs:110-195`, 455). **CONFIRMED** at call-site level; the exact SQL of `get_accessible_channel_ids_cached` was not read (**INFERRED** that it includes open channels, consistent with the write path).
- Community owner ≠ channel owner**. No code path found that maps `relay_members.role` in (`owner`, `admin`) to channel read/write. **CONFIRMED** for the read/write gate. A separate moderation capability grid for community admins exists (`moderation_authz.rs`) and was **not audited in depth** (**UNCLEAR** exactly what admins can do to private-channel content).

16. Message Authorization

`check_channel_membership` (`handlers/ingest.rs:741-770`): *allowed iff the pubkey is an active channel member OR the channel is* `open`. Denial string: `restricted: not a channel member`. Before that, the WS connection itself must have passed NIP-42 + ban check + `relay_members` gate (`handlers/auth.rs:87-241`). Token-scoped access adds `check_token_channel_access` for API tokens. **CONFIRMED**.

Message kinds (from `buzz-core/src/kind.rs`): `text 9` / `40002`, `edit 40003`, `reaction 7`, `presence 20001`, `typing 20002`, `DM open 41010`, `DM add-member 41011`, `DM hide 41012`, `profile 0`. **CONFIRMED**.

17. Authentication vs Authorization

```
AUTHENTICATION ("who is this?")
  private key —signs→ NIP-42 AUTH event (WebSocket)    handlers/auth.rs:87
                      ↳ NIP-98 HTTP event (invites/admin) api/invites.rs:230-265
  result: a verified 32-byte pubkey. No password/OAuth/Okta involved.
  (NIP-FI federated identity exists in buzz-auth but is NOT wired anywhere.)

AUTHORIZATION ("what may this pubkey do?")
  pubkey → ban check (moderation_restriction_state)    auth.rs:120-185
        → [optional pubkey allowlist]                  auth.rs:189-216
        → relay_members row (community membership + role) auth.rs:219-241
          ↳ community-level actions: invite, add/remove/role
        → channel_members row + channels.visibility    ingest.rs:741-770
          ↳ channel-level actions: read, write, admin
        → channel role (owner/admin/member)            channel_authz.rs
```

Relationship required by the audit brief:

```
User identity (pubkey)
  → Community membership (relay_members)
  → Community role (owner|admin|member)
  → Channel membership (channel_members, or open channel)
  → Channel permissions (channel role + visibility)
```

Community role does **not** cascade into channel permissions. **CONFIRMED**.

18. Complete End-to-End Sequence

Owner(A)	A's desktop	Relay	User B's desktop	User B
first launch	Keys::generate() keyring/identity.key (community exists: operator/seed; A is 'owner' in relay_members)		first launch Keys::generate() keyring/identity.key	
"Create invite"	NIP-98 POST /api/invites →	check role∈{owner,admin} 32 random bytes → sha256 → relay_invites		
shares https://host/invite/<code>	← {code,url}		landing page → buzz://join?relay&code	
		machine onboarding (key already exists / import) ← NIP-98 POST /api/invites/claim (B's key) FOR UPDATE; INSERT relay_members(B, 'member', 'invite') publish NIP-43 8000 + 13534 - {status:"joined", role:"member"} →		
		← WS connect + NIP-42 AUTH ban check → relay_members gate ✓ ← REQ / EVENT (kind 9) open channel or channel_members ✓		

19. Identity / Key Lifecycle

```
first launch → Keys::generate() → keyring (or 00600 identity.key)
|
| optional:
| - user imports nsec / ncryptsec [replaces key BEFORE joining]
| - user backs up (NIP-49, password-encrypted, local file only, never sent to a relay)
|
| ↓
join community (claim invite) → pubkey recorded in relay_members (key unchanged)
|
| ↓
daily use: key signs AUTH / NIP-98 / every event
|
| ↓
sign_out → reset sentinel → next boot wipes identity (key lost unless backed up)
```

CONFIRMED (app_state.rs, commands/identity.rs, key_backup.rs:1-12).

20. API / Event / Relay Call Table (OLD BUZZ)

Purpose	Transport	Endpoint / kind	Auth	Handler	Conf.
Mint invite	HTTP POST	/api/invites	NIP-98, role owner/admin	api/invites.rs:284	CONFIRMED
Claim invite	HTTP POST	/api/invites/claim	NIP-98 (joiner)	api/invites.rs:361	CONFIRMED
Accept join policy	HTTP POST	/api/invites/accept-policy	NIP-98	api/invites.rs:199	CONFIRMED
Join policy / terms	HTTP GET	/api/join-policy[/terms /privacy]	none	api/invites.rs:112-147	CONFIRMED
Invite landing page	HTTP GET	/invite/<code>	none (SPA)	router.rs:240	CONFIRMED
Create community	HTTP POST	/operator/communities	NIP-98, operator allowlist	handlers/community_provisioning.rs	CONFIRMED
Add / remove / change role	WS event	kinds 9030 / 9031 / 9032	NIP-42 session	handlers/relay_admin.rs	CONFIRMED
Workspace icon	WS event	kind 9033	NIP-42 session	handlers/relay_admin.rs:262	CONFIRMED
Membership broadcast	relay-signed	kinds 8000 / 8001 / 13534	n/a	side_effects.rs	CONFIRMED
WS authentication	WS	NIP-42 AUTH	signature	handlers/auth.rs:43	CONFIRMED
Join open channel	WS event	kind 9021	NIP-42	side_effects.rs:1950	CONFIRMED
Channel put-user	WS event	kind 9000	NIP-42 + channel role	channel_authz.rs	CONFIRMED

Purpose	Transport	Endpoint / kind	Auth	Handler	Conf.
Messages / reactions	WS event	kinds 9, 40002, 7	NIP-42 + channel gate	ingest.rs:741	CONFIRMED
Presence / typing	WS event	kinds 20001 / 20002	NIP-42	kind.rs:463-467	CONFIRMED
DMs	WS event	kinds 41010 / 41011 / 41012	NIP-42	kind.rs:507-511	CONFIRMED
Profile	WS event	kind 0	NIP-42	commands/profile.rs:40-80	CONFIRMED
Hosted community login	HTTPS	BuilderLab /api/goose/v1/auth/login	BuilderLab account	builderlab.rs	CONFIRMED (client) / UNCLEAR (server)

21. Important Source Files

File (OLD BUZZ)	Why it matters
desktop/src-tauri/src/app_state.rs	Key generation, storage resolution order, recovery states
desktop/src-tauri/src/identity_storage.rs , app_state_keyring.rs	Storage enum, keyring service names
desktop/src-tauri/src/commands/identity.rs	get_nsec , import, backup, sign_out , signing commands
desktop/src-tauri/src/key_backup.rs	NIP-49 encrypted local backup
desktop/src-tauri/src/deep_link.rs , desktop/src/shared/deep-link.ts	buzz://join handling
desktop/src/shared/api/invites.ts	Client mint/claim, NIP-98 header builder
desktop/src/features/onboarding/{communityOnboarding.tsx,useClaimInvite.ts,machineOnboarding.ts}	Stage machines
desktop/src/features/communities/hostedCommunityApi.ts , desktop/src-tauri/src/builderlab.rs	Hosted community / BuilderLab
crates/buzz-relay/src/api/invites.rs	Mint/claim endpoints
crates/buzz-relay/src/handlers/auth.rs	NIP-42, ban, allowlist, membership gates
crates/buzz-relay/src/handlers/{relay_admin.rs,side_effects.rs,ingest.rs,req.rs,channel_authz.rs}	Roles, join, message and read authorization
crates/buzz-relay/src/handlers/community_provisioning.rs	Operator community creation
crates/buzz-core/src/invite.rs , crates/buzz-db/src/store/relay_invite.rs	Code format, mint/claim SQL
migrations/0001_initial_schema.sql , migrations/0025_relay_invites.sql	Membership and invite schemas
crates/buzz-auth/src/nip-fi/*	Dormant OIDC/JWKS federated identity

22. OLD BUZZ Features We Should NOT Replicate

Classification is based on crate descriptions (`Cargo.toml`) and desktop feature-directory contents (non-test file counts in parentheses). Nothing was removed.

Feature	Evidence	Verdict
GitHub / git hosting integration ("projects": repos, PRs, README, clone URL, GitHub mark)	desktop/src/features/projects/* (212 files); git-credential-nostr, git-sign-nostr crates; relay api/git/*	EXCLUDE
Local / client-side agents, managed agents, agent runtime	desktop/src/features/agents (233), desktop/src-tauri/src/managed_agents/*, commands/agent_*.rs, agent-memory, buzz-acp, buzz-agent, sprig, buzz-persona, buzz-dev-mcp	EXCLUDE
Huddles / voice	features/huddle (30), src-tauri/src/huddle, buzz-voice	EXCLUDE
Workflows	features/workflows (48), buzz-workflow, kinds 30620/460xx	EXCLUDE
Terminal / runtime	features/terminal (11)	EXCLUDE
Canvas	kind 40100, commands/canvas.rs	EXCLUDE
Mesh compute / mesh-llm	features/mesh-compute, src-tauri/src/mesh_llm, buzz-relay-mesh	EXCLUDE
Dev tooling / harness	buzz-test-client, buzz-conformance, buzz-admin, buzz-cli, benchmarks, perf, .goose, .codex	EXCLUDE
Mobile push, device pairing	buzz-push-gateway, buzz-pair-relay, buzz-pairing-cli	EXCLUDE (not required for desktop chat)
Nostr identity backup/archive	key_backup.rs, identity-archive, NIP-1A	EXCLUDE for humans (replaced by Okta)
Hosted-community / BuilderLab account	builderlab.rs, HostedCommunity*	EXCLUDE (external SaaS coupling)
Keep (behavior, re-implemented)	communities, community members/roles, channels (open/private), text messages, threads, reactions, DMs, profiles, invites, presence, typing, notifications	REPLICATE behavior only
Borderline, needs a decision	forum (15), pulse (11), reminders (12), search (10), custom emoji (5), GIFs (3), user status (6), moderation (10), channel templates (2)	UNCLEAR: product decision

Remote / server-side agents are in scope per the SWF brief, but as ordinary participants under the SWF architecture (D10), not via OLD BUZZ's local ACP runtime.

23. OLD BUZZ vs SWF BUZZ

SWF current state (read-only inspection)

Area	SWF implementation	Evidence	Conf.
Okta authentication	OIDC Authorization Code + PKCE in Rust; env <code>SWF_BUZZ_OKTA_ISSUER</code> / <code>SWF_BUZZ_OKTA_CLIENT_ID</code> ; custom URL-scheme deep link <code>com.okta.trial-7050986</code>	SWF: <code>src-tauri/src/auth/oidc.rs</code> , <code>tauri.conf.json</code>	CONFIRMED
Login flow	Tauri PKCE exchange → POST <code>/api/session/bootstrap{id_token}</code> → backend verifies ID token via JWKS → upserts users by <code>okta_sub</code> → issues session token	SWF: <code>backend/src/routes/session.rs</code> , <code>jwks.rs</code>	CONFIRMED
Session mechanism	Authorization: Bearer <code><token></code> , server stores only <code>sha256(token)</code> ; token persisted in the OS keychain by Tauri. Docs (D10) say "httpOnly cookie <code>swf_session</code>", but code uses Bearer	<code>backend/src/auth.rs:9-45</code> ; <code>migrations/0001_init.sql</code>	CONFIRMED (doc/code mismatch)
Identity model	<code>users(id, okta_sub UNIQUE, email, display_name)</code> ; email is display-only	<code>migrations/0001_init.sql</code>	CONFIRMED
Membership model	<code>communities, community_members(community_id, user_id, role owner admin member, status)</code>	<code>migrations/0002</code>	CONFIRMED
Invites	POST <code>/api/communities/{id}/invites</code> (create), POST <code>/api/invites/claim</code> , POST <code>/api/invites/{id}/revoke</code> , GET <code>/api/invites/{token}/preview</code> ; 32-byte random, SHA-256 stored, FOR UPDATE claim, TTL 1 h to 30 d, <code>can_manage_invites = owner/admin</code>	<code>routes/mod.rs:33-52</code> , <code>routes/invites.rs</code> , <code>token.rs</code> , <code>migrations/0003</code>	CONFIRMED
Other backend surface	channels + channel members, messages, threads, DMs, realtime <code>/ws</code>	<code>routes/mod.rs</code>	CONFIRMED
Frontend	Both paths present: legacy Nostr (<code>RelayConnectionService</code> , <code>signingService.dev/nip46</code> , <code>nostr-tools</code> in 4 files, <code>useAuth.ts</code> still wires dev/NIP-46 signers) and new HTTP services (<code>*ServiceHttp.ts</code> , <code>InviteService</code> , <code>CommunityService</code> , <code>ApiClient</code>). <code>useAuth.ts</code> states migration is <i>additive</i>	<code>src/features/auth/useAuth.ts:1-70</code>	CONFIRMED; which path each screen actually uses at runtime is UNCLEAR
Session state	Pinia store still models <code>pubkey</code> alongside <code>ApplicationUser</code>	<code>src/stores/session.ts</code>	CONFIRMED

Differences

Topic	OLD BUZZ	SWF BUZZ
Human identity	Nostr pubkey	Okta <code>sub</code> → <code>users.id</code>
Key generation	Client, first launch	None (by design)
HTTP auth	NIP-98 per request	Bearer session token

Topic	OLD BUZZ	SWF BUZZ
WS auth	NIP-42	Session bearer token in a <code>?token=</code> query parameter on <code>/ws</code> (<code>routes/realtime.rs:3-9,35-46</code> ; the file's own comment notes the token travels in the URL)
Tenancy	Host header → <code>communities</code>	<code>community_id</code> in path
Community creation	Operator / BuilderLab	POST <code>/api/communities</code> by any authenticated user (creator becomes owner, <code>postgres.rs:134</code>)
Invite mint route	POST <code>/api/invites</code>	POST <code>/api/communities/{id}/invites</code>
Invite TTL floor / cap	60 s / 30 d	1 h / 30 d
Invite use cap	1..10 000	≥ 1, no upper bound
Join policy (terms/age)	Supported	Not present
Membership broadcast	NIP-43 events	None
Claim rate limiting	10 / min / pubkey	None (grep of <code>backend/src</code> finds no rate-limit code)
Claim role	<code>member</code> (DB CHECK forbids anything else)	<code>member</code> (SQL literal in the claim transaction, <code>repo/postgres.rs ~817-819</code>); not additionally DB-constrained
Claim idempotency	<code>already_member</code> returned without consuming a use	Same: <code>AlreadyMember</code> is returned <i>before</i> <code>used_count</code> is incremented (<code>postgres.rs ~776-840</code>)
Invite revoke	Not found in relay path (UNCLEAR)	Present

SWF parts that depend on Okta

`src-tauri/src/auth/oidc.rs`, `commands/auth.rs`, `commands/secure_storage.rs`, `tauri.conf.json` (deep-link scheme), `backend/src/jwks.rs`, `config.rs`, `routes/session.rs`, `models.rs`, `repo/*` (`okta_sub`), and frontend `features/auth/authService.okta*.ts`, `oktaBrowserFlow.ts`, `useAuth.ts`, `stores/session.ts`, `views/LoginView.vue`, `OktaCallbackView.vue`, `InviteLandingView.vue`, `communities/pendingInvite.ts`, `services/ApiClient.ts`, `RelayConnectionService.ts`, `.env` keys `VITE_OKTA_*` / `SWF_BUZZ_OKTA_*`.

What would need disabling if OLD BUZZ identity behavior were adopted (not recommended; documented only)

- Backend: `jwks.rs`, POST `/api/session/bootstrap`, `users.okta_sub` (would become `pubkey`), `AuthUser` extractor.
- Tauri: `auth/oidc.rs`, `commands/auth.rs`, the Okta deep link in `tauri.conf.json`.
- Frontend: all `authService.okta*`, `OktaCallbackView`, Okta login button; would re-enable `signingService.dev / nip46` and pubkey-keyed `session` fields.
- Every such change would contradict D10 and the standing rule "no Nostr human identity".

24. What SWF BUZZ Would Need to Reproduce Later (behavior only)

Confirmed OLD BUZZ behaviors worth matching, expressed in the SWF (`user_id`) model:

1. **Owner/admin-only invite minting**, opaque single-secret code, hash-only storage, community bound to the code. (SWF already does this.)
2. **Atomic claim**: row lock, expiry/exhaustion checks, **idempotent** `already_member` **that does not consume a use**, role pinned to `member`. (SWF already matches: `FOR UPDATE`, `AlreadyMember` before increment, `'member'` literal. Only the DB-level CHECK that OLD BUZZ uses to forbid elevated invite roles is absent.)
3. **Claim rate limiting** per authenticated user (OLD: 10/min). (Gap: SWF has none.)
4. **Landing** → **open-in-app** → **login** → **claim** with the invite restored after login (SWF's `pendingInvite.ts` and `/invite/<token>` route appear to cover this; not audited).
5. **Membership gate before any channel data** and **channel-level authz separate from community role** (community owner does not automatically read private channels).
6. **Open vs private channel semantics**: self-join only for open channels; private requires an invitation.
7. **Member joins/leaves surfaced** (OLD uses NIP-43 events; SWF would use its own realtime `/ws`).
8. **Optional join-policy acceptance** (terms/age): only if the business needs it.
9. **Last-owner protection** for channels and communities.
10. Feature behaviors: text messages, threads, reactions, DMs (open/hide), presence, typing, profiles, profile → DM.

25. What SWF BUZZ Should NOT Reproduce

- Client-generated Nostr keys, `nsec` / `npub`, NIP-49 backups, key import/export UI, sign-out-wipes-identity.
 - NIP-42 / NIP-46 / NIP-98 / bunker as human authentication or invite authentication.
 - `relay_members.pubkey` / `channel_members.pubkey` as the human identifier.
 - Trusting any client-supplied user id in invite create/claim; the backend must derive the user from the session (already the SWF design).
 - BuilderLab hosted-community coupling; operator-only community creation (unless SWF wants that policy).
 - Everything listed in §22 as EXCLUDE (GitHub/projects, local agents, huddles, workflows, terminal, canvas, mesh, mobile push, pairing).
 - The dormant NIP-FI module (it binds an OIDC identity **to a Nostr key**, the opposite of D10).
-

26. Open Questions / Unknowns

1. **BuilderLab server behavior** (community provisioning, identity binding): external, not in this repo. **UNCLEAR**.
2. **How a newly joined user lands in the Welcome channel** (auto-join mechanism). Not traced.
3. **Exact powers of community admins under** `moderation_authz.rs` (e.g., over private-channel content). Not fully audited.
4. `get_accessible_channel_ids` **SQL** (whether open channels are always included). Consistent with the write path but not read.
5. **Whether OLD BUZZ has an invite revoke path** in the relay HTTP API (none found in `api/invites.rs`; DB has expiry/`max_uses` only).
6. **SWF: which frontend screens still run on the Nostr path at runtime** versus the new HTTP services.
7. **SWF documentation mismatch**: D10 says an httpOnly `swf_session` cookie; the code uses a Bearer header (and a `?token=` URL parameter for `/ws`). Decide which is intended and correct the docs or the code.

8. **SWF: who may create a community.** Confirmed: `POST /api/communities` needs only an authenticated session (`AuthUser`); the creator becomes `owner`. Confirm this is the intended policy (OLD BUZZ restricts creation to operators).
9. **Deployment:** whether a production SWF relay is still required at all for agents (D10 keeps agents on Nostr).

27. Recommended Implementation Order for the FUTURE

Recommendations only. Nothing here has been implemented.

1. **Decide the target boundary first.** Confirm D10 stands (Okta → Application User, no human Nostr) and that agents remain the only Nostr participants. Resolve open questions 6-8 with a short SWF code trace.
2. **Freeze the reference behavior** from §24 as acceptance tests (invite mint/claim idempotency, role pin, rate limit, access matrix: community role × channel membership × visibility).
3. **Close SWF backend parity gaps** (claim rate limiting, community-creation policy, optional DB-level role CHECK on invites, doc fix for Bearer vs cookie, and moving the `/ws` token out of the URL if that is a concern).
4. **Finish the HTTP migration feature by feature** (communities → invites → channels → messages/threads → DMs → presence/typing/reactions), pointing each UI at `*ServiceHttp` and deleting its Nostr twin *after* verification (D10's own sequencing).
5. **Wire Okta invite round-trip end to end** (`/invite/<token>` → Okta login → restore URL → claim) and test with two real users.
6. **Then, and only then, remove dead Nostr-human code** (signing services, pubkey session fields, `nostr-tools` from human paths).
7. **Explicitly exclude** the §22 features; do not import their code.
8. **Decide the borderline features** (forum, pulse, reminders, search, moderation, emoji/GIFs, status) as product scope before any work.

Audit scope note: no application source, package, Cargo, environment, Okta, or database file was modified. The only files created are this report and its PDF rendering in `swf buzz/docs/`.